

```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
from itertools import combinations
```

```
def get_support_count(itemset, transactions):
    """Counts in how many transactions the given itemset appears."""
    count = 0
    for t in transactions:
        if itemset.issubset(t):
            count += 1
    return count
```

```
def get_support_count(itemset, transactions):
    """Counts in how many transactions the given itemset appears."""
    count = 0
    for t in transactions:
        if itemset.issubset(t):
            count += 1
    return count
```

```
def generate_C1(transactions):
    """Generate candidate 1-itemsets (C1) - every unique item found in the data."""
    items = set()
    for t in transactions:
        for item in t:
            items.add(frozenset([item]))
    return items
```

```
def generate_C1(transactions):
    """Generate candidate 1-itemsets (C1) - every unique item found in the data."""
    items = set()
    for t in transactions:
        for item in t:
            items.add(frozenset([item]))
    return items
```

```
def generate_C1(transactions):
    """Generate candidate 1-itemsets (C1) - every unique item found in the data."""
    items = set()
    for t in transactions:
        for item in t:
            items.add(frozenset([item]))
    return items
```

```
def prune(candidates, transactions, min_support_count):
    """Calculate support count for each candidate and keep only those
    meeting the minimum support count. Returns a dict {itemset: count}."""
    frequent = {}
    for c in candidates:
        cnt = get_support_count(c, transactions)
        if cnt >= min_support_count:
            frequent[c] = cnt
    return frequent

def generate_next_candidates(prev_frequent_itemsets, k):
    """Join step: combine frequent (k-1)-itemsets to form candidate k-itemsets.
    Then apply the Apriori prune step: discard any candidate that has a
    (k-1)-subset which is NOT in the previous frequent itemsets."""
    prev_items = list(prev_frequent_itemsets.keys())
    candidates = set()

    # JOIN step
    for i in range(len(prev_items)):
        for j in range(i + 1, len(prev_items)):
            union_set = prev_items[i] | prev_items[j]
            if len(union_set) == k:
                candidates.add(union_set)

    # PRUNE step (Apriori property: all (k-1)-subsets must be frequent)
    pruned_candidates = set()
```

```

for c in candidates:
    subsets = combinations(c, k - 1)
    valid = True
    for s in subsets:
        if frozenset(s) not in prev_frequent_itemsets:
            valid = False
            break
    if valid:
        pruned_candidates.add(c)

return pruned_candidates

```

```

def run_apriori(transactions, min_support_count, label):
    """Runs the full Apriori process level by level and prints every step."""
    transactions = [set(t) for t in transactions]
    print(f"\nTotal Transactions for {label}: {len(transactions)}")
    print(f"Minimum Support Count Required: {min_support_count}")

    all_frequent_itemsets = {}
    k = 1

    # ---- Step 1: C1 and L1 ----
    C1 = generate_C1(transactions)
    print(f"\n--- C1 (Candidate 1-itemsets) ---")
    for c in C1:
        print(f"{set(c)} : support count = {get_support_count(c, transactions)}")

    L1 = prune(C1, transactions, min_support_count)
    print(f"\n--- L1 (Frequent 1-itemsets after pruning) ---")
    for itemset, cnt in L1.items():
        print(f"{set(itemset)} : {cnt}")

    all_frequent_itemsets.update(L1)
    prev_L = L1
    k = 2

    # ---- Step 2 onward: Ck and Lk until no more frequent itemsets found ----
    while prev_L:
        Ck = generate_next_candidates(prev_L, k)
        if not Ck:
            break

        print(f"\n--- C{k} (Candidate {k}-itemsets after join + prune step) ---")
        for c in Ck:
            print(f"{set(c)} : support count = {get_support_count(c, transactions)}")

        Lk = prune(Ck, transactions, min_support_count)
        print(f"\n--- L{k} (Frequent {k}-itemsets after pruning) ---")
        if Lk:
            for itemset, cnt in Lk.items():
                print(f"{set(itemset)} : {cnt}")
        else:
            print("No frequent itemsets at this level.")

        all_frequent_itemsets.update(Lk)
        prev_L = Lk
        k += 1

    return all_frequent_itemsets

```

```

def print_summary(all_frequent_itemsets, total_transactions):
    """Prints final summary table with support count and support fraction."""
    print("\n=== FINAL FREQUENT ITEMSETS SUMMARY ===")
    for itemset, cnt in all_frequent_itemsets.items():
        support_fraction = cnt / total_transactions
        print(f"{set(itemset)} -> support count = {cnt}, support = {support_fraction:.3f}")

```

```

# -----
# Question 1: Hospital Pharmacy Analysis
# -----

print("=" * 60)
print("QUESTION 1: HOSPITAL PHARMACY ANALYSIS")
print("=" * 60)

```

```

pharmacy_data = [
    ['Paracetamol', 'Vitamin C', 'Cough Syrup'],
    ['Paracetamol', 'Antibiotic', 'Vitamin C'],
    ['Antibiotic', 'Vitamin C', 'Pain Reliever'],
    ['Paracetamol', 'Vitamin C', 'Pain Reliever'],
    ['Paracetamol', 'Antibiotic', 'Vitamin C'],
    ['Vitamin C', 'Cough Syrup']
]

min_support_count = 3 # given in the question

pharmacy_frequent = run_apriori(pharmacy_data, min_support_count, "Pharmacy Data")
print_summary(pharmacy_frequent, len(pharmacy_data))

```

```

=====
QUESTION 1: HOSPITAL PHARMACY ANALYSIS
=====

Total Transactions for Pharmacy Data: 6
Minimum Support Count Required: 3

--- C1 (Candidate 1-itemsets) ---
{'Vitamin C'} : support count = 6
{'Antibiotic'} : support count = 3
{'Cough Syrup'} : support count = 2
{'Pain Reliever'} : support count = 2
{'Paracetamol'} : support count = 4

--- L1 (Frequent 1-itemsets after pruning) ---
{'Vitamin C'} : 6
{'Antibiotic'} : 3
{'Paracetamol'} : 4

--- C2 (Candidate 2-itemsets after join + prune step) ---
{'Antibiotic', 'Paracetamol'} : support count = 2
{'Antibiotic', 'Vitamin C'} : support count = 3
{'Vitamin C', 'Paracetamol'} : support count = 4

--- L2 (Frequent 2-itemsets after pruning) ---
{'Antibiotic', 'Vitamin C'} : 3
{'Vitamin C', 'Paracetamol'} : 4

=== FINAL FREQUENT ITEMSETS SUMMARY ===
{'Vitamin C'} -> support count = 6, support = 1.000
{'Antibiotic'} -> support count = 3, support = 0.500
{'Paracetamol'} -> support count = 4, support = 0.667
{'Antibiotic', 'Vitamin C'} -> support count = 3, support = 0.500
{'Vitamin C', 'Paracetamol'} -> support count = 4, support = 0.667

```

```

# -----
# Question 2: University Library Book Borrowing Analysis
# -----

print("\n" + "=" * 60)
print("QUESTION 2: LIBRARY BOOK BORROWING ANALYSIS")
print("=" * 60)

library_data = [
    ['Data Structures', 'Python Programming', 'DBMS'],
    ['Data Structures', 'Operating Systems'],
    ['Python Programming', 'DBMS', 'Machine Learning'],
    ['Data Structures', 'Python Programming', 'Machine Learning'],
    ['Data Structures', 'Python Programming', 'DBMS'],
    ['Python Programming', 'DBMS']
]

library_frequent = run_apriori(library_data, min_support_count, "Library Data")
print_summary(library_frequent, len(library_data))

```

```

=====
QUESTION 2: LIBRARY BOOK BORROWING ANALYSIS
=====

Total Transactions for Library Data: 6
Minimum Support Count Required: 3

--- C1 (Candidate 1-itemsets) ---
{'Python Programming'} : support count = 5
{'DBMS'} : support count = 4
{'Operating Systems'} : support count = 1
{'Data Structures'} : support count = 4
{'Machine Learning'} : support count = 2

--- L1 (Frequent 1-itemsets after pruning) ---
{'Python Programming'} : 5

```

```
{'DBMS'} : 4
{'Data Structures'} : 4

--- C2 (Candidate 2-itemsets after join + prune step) ---
{'Data Structures', 'Python Programming'} : support count = 3
{'Data Structures', 'DBMS'} : support count = 2
{'DBMS', 'Python Programming'} : support count = 4

--- L2 (Frequent 2-itemsets after pruning) ---
{'Data Structures', 'Python Programming'} : 3
{'DBMS', 'Python Programming'} : 4

=== FINAL FREQUENT ITEMSETS SUMMARY ===
{'Python Programming'} -> support count = 5, support = 0.833
{'DBMS'} -> support count = 4, support = 0.667
{'Data Structures'} -> support count = 4, support = 0.667
{'Data Structures', 'Python Programming'} -> support count = 3, support = 0.500
{'DBMS', 'Python Programming'} -> support count = 4, support = 0.667
```

Start coding or [generate](#) with AI.